

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead



UNIT III · IoT PLATFORMS, EDGE & CLOUD

Topic 2

Hardware platforms for connected devices

UNIT III · TOPIC 2

Hardware Platforms

Arduino, ESP32, Raspberry Pi, NodeMCU, and what plugs into them



Arduino | ESP32 | Raspberry Pi | NodeMCU

1 IoT hardware platforms

The physical board that senses the world and runs the code

In Simple Words

A hardware platform is the physical board that reads sensors, runs your code, and drives actuators. Every board trades compute power, connectivity, power draw, and cost differently — so “which board?” is one of the first real decisions in any IoT project.

Four boards run through this whole topic: Arduino, ESP32, Raspberry Pi, and NodeMCU.

What Every Choice Trades Off

Compute power

From a simple 8-bit loop to a full quad-core Linux machine.

Connectivity

Some boards need an add-on module; others have Wi-Fi built in.

Power draw

Microamps in deep sleep versus an amp for a full computer.

Cost

A few dollars for a bare board to tens of dollars for a Pi.

2 Where hardware platforms fit

Unit III’s mental model, with Hardware in focus



Hardware

this topic →

Senses the world and runs the code, physically



Edge

Makes the fast decisions that can’t wait



Cloud

Stores, aggregates and analyzes at scale



AI

Learns the pattern behind all of it



Hardware is the foundation

Every other layer depends on it: no board, no sensing, no local decision, and nothing to send to the cloud in the first place.

Arduino: the simple, deterministic workhorse

An 8-bit microcontroller with no operating system at all

MCU

ATmega328P, 8-bit AVR @ 16 MHz

Memory

2 KB RAM, 32 KB flash

Connectivity

None built in — add a shield or module

Power

≈ 20–50 mA active; no deep-sleep tricks

Best for

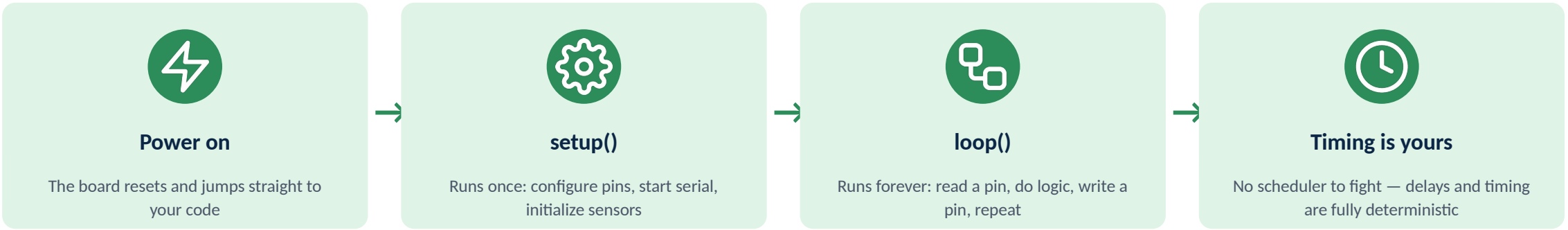
- Learning the fundamentals of embedded code
- Deterministic timing — no OS jitter to fight
- Simple sensor and actuator control loops
- Projects that never need to touch a network

For IoT specifically, it usually needs help: pair it with a Wi-Fi or LoRa module to actually get data off the board.

3

How an Arduino sketch actually runs

No operating system — just your code, in an infinite loop



Why this matters for IoT

- No multitasking — one slow sensor read blocks everything else in the loop
- No memory manager — you allocate almost everything at compile time
- No network stack by default — that’s bolted on via a shield or library
- This simplicity is exactly why timing-critical control code prefers it

ESP32: the wireless IoT workhorse

Dual-core, with Wi-Fi and Bluetooth built right in

MCU

Dual-core Xtensa LX6, up to 240 MHz

Memory

520 KB SRAM, 4 MB+ flash

Connectivity

Wi-Fi and Bluetooth (BLE + Classic) built in

Power

≈ 160–260 mA on Wi-Fi TX; < 10 µA in deep sleep

Best for

- Battery-powered sensor nodes that wake, send, and sleep
- Projects that need Wi-Fi or BLE without an add-on module
- Lightweight edge logic — two cores means room to spare
- The de facto default board for new IoT projects today

Recall from Unit II: this is the board behind many of our short-range and networking examples — an ESP32 sending readings over Wi-Fi.

Raspberry Pi: a full computer on a board

Real Linux, real compute — at a real power cost

CPU

Quad-core ARM Cortex-A72, up to 1.8 GHz

Memory

2–8 GB RAM (model-dependent)

Connectivity

Wi-Fi, Bluetooth, Ethernet, USB, HDMI

Power

≈ 600 mA–1.2 A — far more than any microcontroller

Best for

- An edge gateway that aggregates many smaller nodes
- Running real applications, a database, or a web dashboard
- Computer vision and edge AI inference
- Anywhere “just run Linux” beats squeezing onto a microcontroller

A Raspberry Pi is not a microcontroller wearing a costume — it is a real computer, with a real OS, real power draw, and real capability.

$ NodeMCU: the budget Wi-Fi board

An ESP8266 dev board built for cheap, simple Wi-Fi projects

MCU

Single-core Tensilica, ≈ 80–160 MHz (ESP8266)

Memory

≈ 50 KB usable RAM

Connectivity

Wi-Fi built in — no Bluetooth on the original chip

Power

Similar order to ESP32, fewer sleep tricks

Best for

- The cheapest way to get a sensor onto Wi-Fi
- Home-automation projects at hobby scale
- Simple, single-purpose nodes with few pins needed
- Programmed with the same Arduino IDE and C++ as Arduino

Think of it as ESP32's cheaper, simpler cousin: less compute, no Bluetooth, but enough Wi-Fi to get most hobby projects online.

4 Sensors and modules: how they connect

Every board talks to the outside world through one of a handful of interfaces

GPIO

On / off, digital

Buttons, relays, LEDs — one wire, one signal.

Analog (ADC)

A voltage, read as a number

Potentiometers, simple analog temperature sensors.

I2C

Two wires, many devices

Each chip has an address — e.g. a BME280 and an OLED on the same bus.

SPI

Four wires, high speed

SD cards, fast displays — more wires, more bandwidth.

UART

Simple serial, point-to-point

GPS modules, talking to another microcontroller.

PWM

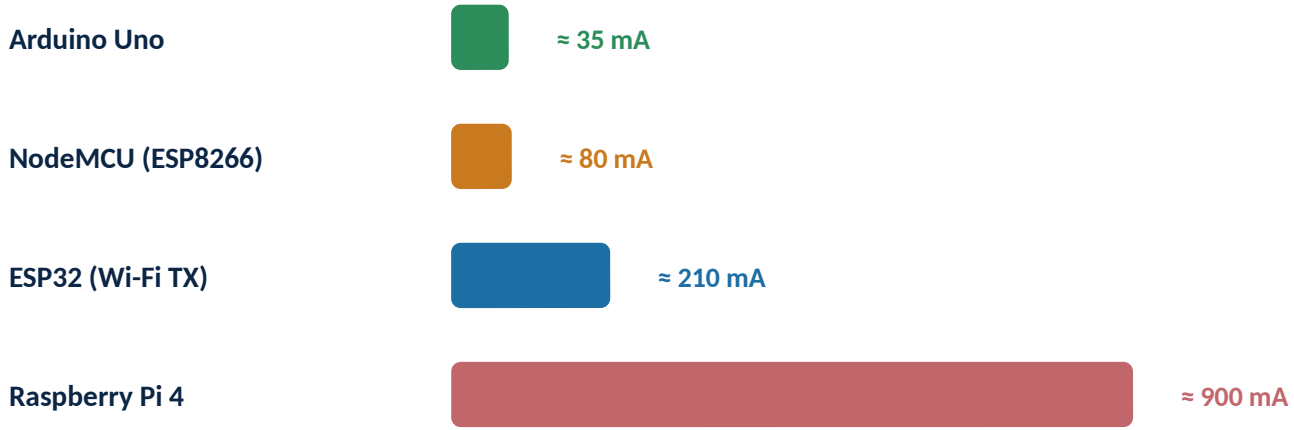
A rapidly-switched digital pin

Motor speed, servo angle, LED brightness.

5 Compute power and power draw pull in opposite directions

The single trade-off that explains almost every board choice

Illustrative active-mode current draw by board (mA)



Illustrative, not a benchmark

Exact figures vary by board revision and workload, but the direction never does: more cores, more RAM and an operating system always cost more current.

The pattern: Arduino sips power but can't think much; a Raspberry Pi thinks a lot but drains a battery in hours, not months. ESP32 and NodeMCU sit in between — which is exactly why they dominate battery-powered IoT.

6 Which board should I use?

A first-pass guide from need to board

If your need is...	Board
A classroom project learning embedded basics Deterministic timing, huge community, no networking to confuse things.	Arduino
A battery sensor that wakes, sends, and sleeps Wi-Fi and BLE on-chip, with genuinely low deep-sleep current.	ESP32
An edge gateway running real software Full Linux, real compute, real storage.	Raspberry Pi
The cheapest way to get one sensor onto Wi-Fi Minimal cost, familiar Arduino-style coding.	NodeMCU

A starting point, not a rule: real projects often combine several boards in one deployment.

Hardware platforms, side by side

Typical values only — always check the datasheet for your exact board

Board	CPU	Memory	Connectivity	Power	Cost
Arduino Uno	8-bit @ 16 MHz	2 KB RAM	None built in	≈ 20–50 mA	$
ESP32	Dual-core @ 240 MHz	520 KB RAM	Wi-Fi + Bluetooth	≈ 160–260 mA	$
Raspberry Pi 4	Quad-core @ 1.8 GHz	2–8 GB RAM	Wi-Fi, BT, Ethernet	≈ 600 mA–1.2 A	$$$
NodeMCU	Single-core @ ≤160 MHz	≈ 50 KB RAM	Wi-Fi only	≈ 80–200 mA	$

No single board wins everywhere: match the board to the compute and power the job actually needs.

The big picture

Four boards, one job: sense the world and run the code

Arduino	Simple, deterministic, no networking of its own — great for learning.
ESP32	Wi-Fi and Bluetooth on-chip, low deep-sleep power — today's default.
Raspberry Pi	A full Linux computer — real compute, real power draw.
NodeMCU	The cheapest way onto Wi-Fi, styled like Arduino to code.
Up next	Topic 3: Cloud Computing for IoT — where all this data goes to scale.



One-line takeaway

“The best board is the one whose compute, connectivity, power draw and cost actually match the job in front of you.”

Arduino · ESP32 · Raspberry Pi · NodeMCU

Up next: Topic 3 — Cloud Computing for IoT (architecture, device management, storage, analytics, dashboards)

1

Cloud computing for IoT

Where millions of readings become one coherent picture

In Simple Words

The cloud is where a fleet of devices becomes one coherent system: a place to register every device, store its history, run analysis no single board could manage, and show a human what is actually happening — across ten devices or ten million.

Five pieces run through this whole topic: architecture, device management, storage, analytics, dashboards.

What Every Choice Trades Off

Latency vs. scale

A quick local check is fast; the cloud is slower but sees everything.

Cost per device

Every message, byte stored and query runs up a bill — multiplied by fleet size.

Security surface

Every connected device is a door that has to be locked and verified.

Vendor lock-in

Managed services are convenient, but switching providers later is hard.

2 Where cloud fits

Unit III's mental model, with Cloud in focus



Hardware

Senses the world and runs the code, physically



Edge

Makes the fast decisions that can't wait



Cloud

this topic →

Stores, aggregates and analyzes at scale



AI

Learns the pattern behind all of it

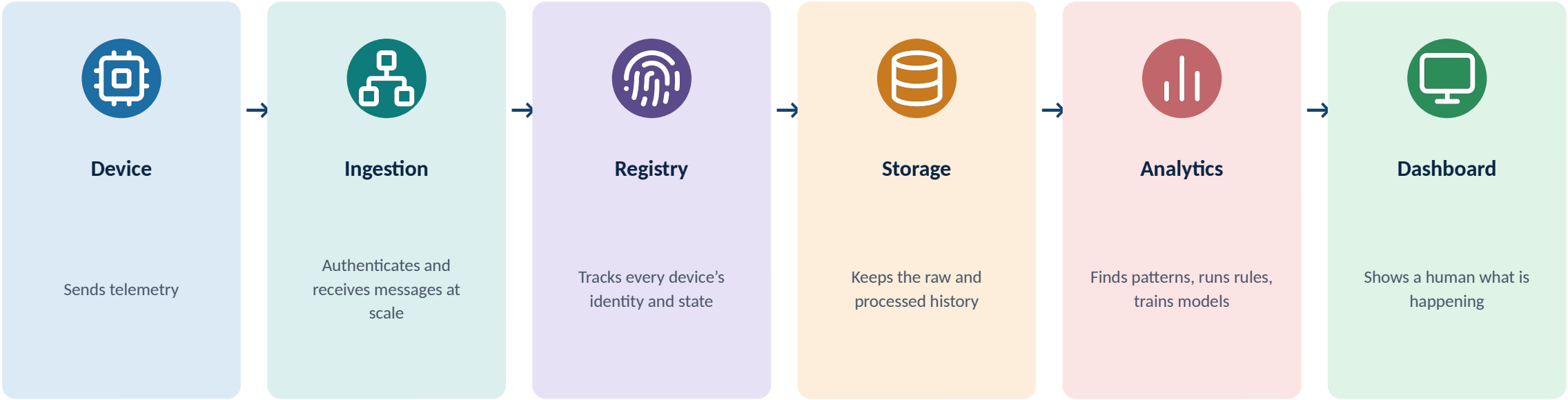


Cloud is the memory and the meeting point

Hardware and edge live in the moment. The cloud remembers everything, and is where every device's data meets every other device's data.

IoT cloud architecture: the big picture

Six stages between a device and a decision



Providers package this differently — AWS IoT Core, Azure IoT Hub, Google Cloud IoT — but every one of them implements roughly this same six-stage shape.

3

How one reading actually flows through the cloud

Following a single message from device to dashboard



1. Publish

Device sends a reading, usually over MQTT, with its own credentials



2. Authenticate

The cloud verifies the device's identity before accepting anything



3. Route

A rules engine decides where this message needs to go



4. Store

The reading lands in a time-series database or data lake



5. Display

A dashboard queries the latest value and redraws

All five stages typically finish in well under a second — but each one is a real piece of infrastructure someone has to design and pay for.

Device management: knowing what's out there

Every device needs an identity, a state, and a way to be updated

Provisioning & identity

Every device is issued its own credentials — often an X.509 certificate — at setup, so the cloud can tell it apart from every other device.

Device shadow / twin

A cloud-side JSON snapshot of a device's desired and last-reported state, so apps can read or set it even while the device is offline.

OTA updates

New firmware ships to devices over the air, often to a small batch first, so one bad update doesn't brick the whole fleet.

Fleet monitoring

Connectivity, battery and error status roll up into one view, so a failing device gets noticed before it goes fully silent.

Data storage: not all readings age the same way

Hot, frequently-queried data and cold, cheaply-archived data need different homes

Hot storage

What

A time-series database tuned for fast recent-data queries

Holds

The last hours or days, ready for a live dashboard

Cost

Higher per byte — built for speed, not bulk

Example

InfluxDB, TimescaleDB, managed time-series services

Cold storage

What

Object storage or a data lake for bulk history

Holds

Months or years, queried in batches, not live

Cost

Much cheaper per byte — built for bulk, not speed

Example

S3-style object storage, archival data lakes

A common pattern: keep raw readings hot for days, then age them into cheaper cold storage as aggregated summaries.

Cloud analytics: turning history into insight

Two speeds of analysis, doing very different jobs

Stream processing

When

As each message arrives, in near real time

Does

Rules engines: “if temperature > 80, alert”

Latency

Seconds or less

Use

Safety alerts, live thresholds, fraud checks

Batch processing

When

On a schedule — hourly, nightly, weekly

Does

Aggregates, trains models, builds reports

Latency

Minutes to hours — that’s fine

Use

Weekly summaries, predictive maintenance models

Stream processing catches problems as they happen; batch processing finds the patterns you’d never see in a single reading.

IoT dashboards: where a human finally looks

The payoff for everything upstream — if it's designed well

Real-time view

Live gauges and maps, refreshing as new readings arrive.

Historical view

Trends over days or months — spot the slow drift, not just the spike.

Alerts

A perfectly timed notification beats a dashboard nobody is watching.

Role-based access

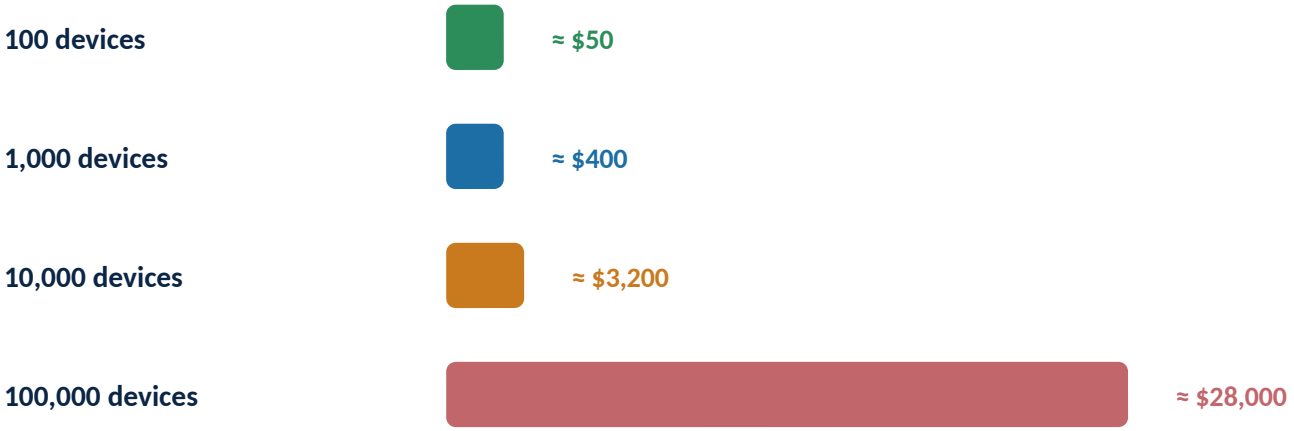
An operator, a manager and an auditor rarely need to see the same screen.

How successful a deployment feels to its users depends heavily on this last layer, even when every layer beneath it is technically flawless.

4 Cost scales with device count

The trade-off that shapes almost every cloud decision in IoT

Illustrative monthly cloud cost by fleet size (USD)



Illustrative, not a quote

Real pricing depends on message size, frequency, storage tier and provider — but connections, messages and storage all multiply by device count.

This is exactly why device management and hot/cold storage tiers exist: a sloppy design that's fine for 100 devices can become very expensive at 100,000.

5

Which cloud pattern should I use?

A first-pass guide from need to pattern

If your need is...	Pattern
Get telemetry in reliably, at scale A managed ingestion layer authenticates and absorbs bursts for you.	Device gateway / IoT Hub
React the instant one device misbehaves Rules evaluate each message as it lands, in near real time.	Stream processing
Build a weekly maintenance report Latency doesn't matter; a scheduled job can crunch it all at once.	Batch analytics
Keep years of history cheaply Bulk storage, queried rarely, beats a database built for speed.	Cold storage / data lake

A starting point, not a rule: most real systems run several of these patterns side by side.

Cloud computing concepts, side by side

Concept · role · a concrete example

Concept	Role	Example
Architecture	The six-stage pipeline from device to dashboard	AWS IoT Core, Azure IoT Hub
Device mgmt	Identity, provisioning, OTA updates, fleet health	Device shadow / digital twin
Storage	Hot time-series data vs. cheap cold archives	InfluxDB, S3-style object storage
Analytics	Stream (real-time rules) vs. batch (scheduled jobs)	Threshold alerts, nightly reports
Dashboards	Where a human sees live and historical status	Live gauges, role-based views

Five pieces, one job: turn a fleet of individual devices into one system you can see, trust, and act on.

The big picture

Cloud in five rows: how a fleet becomes one system

Architecture	Device to dashboard in six stages — ingest, register, store, analyze, show.
Devices	Every device gets an identity, a shadow, and a way to be updated.
Storage	Hot for what's live, cold for what's history — cost follows the split.
Analytics	Stream to react now; batch to see the pattern later.
Up next	Topic 4: Edge Computing — the trade-off we kept previewing, in full.



One-line takeaway

"The cloud turns a fleet of lonely devices into one system — if you design its identity, storage, and analysis to scale with it."

Architecture · Devices · Storage · Analytics · Dashboards

Up next: Topic 4 — Edge Computing (need for edge, edge vs cloud, real-time processing, edge AI, fog computing)

Edge Computing Brings the Processing to the Data

Instead of sending every reading to a distant data center, edge computing processes it right where it is created.

Compute at the source

Milliseconds, not seconds

In Simple Words

A sensor produces data every second. Sending all of it to the cloud and waiting for an answer is slow, expensive and fragile.

Edge computing places a small amount of that cloud-like intelligence inside or near the device itself, so it can decide and act immediately.



No round trip to a distant server needed for the decision that matters right now.

Why It Matters

Latency kills real-time control

A cloud round trip can take hundreds of milliseconds — too slow for braking or safety shut-offs.

Bandwidth is not free

Streaming every raw reading from millions of devices is expensive and often unnecessary.

Connectivity is not guaranteed

Factories, farms and vehicles lose signal. Local processing keeps working anyway.

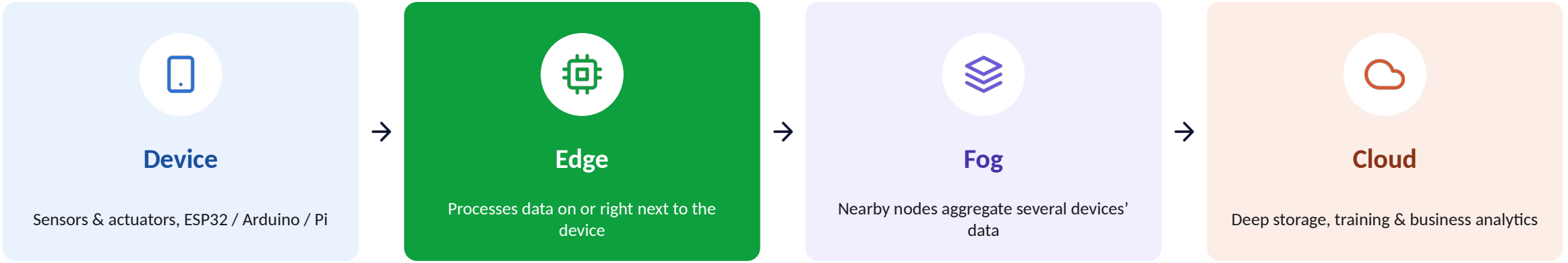
Some data should stay local

Raw video or biometrics can be filtered on-device before anything leaves the site.

Edge computing does not replace the cloud — it decides what needs an instant local answer and what can wait.

1 Recap: where edge computing lives

Unit III so far — hardware platforms and the cloud. Edge sits between them.



Topic 4 lives at the Edge (and touches Fog)

In Unit III Topic 2 we met the boards (ESP32, Arduino, Pi). In Topic 3 we saw the cloud handle storage and heavy analytics. This topic is about the layer in between: what runs locally, right where the sensor is, and why that layer exists at all.

Almost all of this topic is about what happens between the device and the data center.

2 Why We Need Edge Computing

Four forces push processing out of the cloud and toward the device.

Latency

Safety and control loops need answers in milliseconds. A cloud round trip is too slow for a robot arm or a braking system.

Bandwidth cost

A single camera can generate gigabytes per hour. Sending it all to the cloud does not scale to millions of devices.

Unreliable connectivity

Remote farms, ships, mines and factories lose network access. Local processing keeps the system working offline.

Privacy & compliance

Faces, voices and health signals can be filtered or anonymized on-device before anything ever leaves the building.

None of these forces alone justifies edge computing everywhere — together, they make it essential somewhere in almost every real deployment.

3 Edge vs Cloud: The Trade-off

Neither wins outright — each is strong where the other is weak.

Edge

Fast

Answers in milliseconds — no network hop

Works offline

Keeps running when connectivity drops

Private

Sensitive raw data never has to leave the site

Cheap to run

No per-byte cloud transfer or storage cost

VS

Cloud

Unlimited power

Trains large models on years of accumulated data

Full history

Stores everything for audits and long-term trends

Global view

Sees patterns across every device, every site

Easy to update

One place to improve the model for everyone

Most real systems use both: quick decisions at the edge, deep learning and long-term storage in the cloud.

The cost of the round trip

<10 ms

typical edge decision time

runs on the device itself

~100–300 ms

typical cloud round trip

sensor → internet → server → back

30x+

faster response at the edge

for a simple local decision

Why the gap matters

A cloud round trip is not one hop — it is a chain: device → gateway → internet → load balancer → server → database → and all the way back. Every hop adds queueing and network delay on top of the actual processing time. For a dashboard update, a few hundred milliseconds is invisible. For a robot arm, an anti-lock brake, or a safety shut-off, it can be the difference between a near-miss and an accident. Edge computing removes most of that chain by answering the question on the spot.

This is exactly why latency-sensitive control loops are almost always pushed to the edge.

4 Real-Time Processing

Some decisions have a deadline — being right a second late is the same as being wrong.

What "real-time" means here

Real-time does not mean "fast" — it means the answer must arrive within a fixed deadline, every single time, or the outcome is treated as a failure.

Hard real-time

Missing the deadline is unsafe — e.g. an airbag or an anti-lock brake

Soft real-time

Missing the deadline degrades quality but is not dangerous — e.g. a video call

Why it must happen at the edge

Industrial safety shut-off

Stop a machine within milliseconds of detecting a fault

Autonomous vehicle braking

Decide to brake before a cloud request could even leave the car

Predictive maintenance alarms

Flag an abnormal vibration the instant a threshold is crossed

A cloud dependency turns a millisecond deadline into a network gamble — the edge removes that gamble.

5 Edge AI

Trained models are shrunk to run inference directly on the device itself.

Train

Large model trained in the cloud on lots of data

→

Compress

Model is quantized and shrunk to fit a tiny chip

→

Deploy

Compact model is pushed onto the edge device

→

Infer

Device makes predictions locally, in real time

Where you have already seen this

Keyword spotting

A smart speaker recognizing "Hey ___" without calling the cloud

Vibration anomaly detection

A motor sensor flagging unusual vibration patterns locally

Vision on a camera module

Detecting a person or object directly in the camera chip

This shrink-and-deploy pattern is often called TinyML — machine learning small enough for a microcontroller.

6 Fog Computing

A middle tier between many edge devices and the distant cloud.

What a fog node does

Aggregates

Collects data from many nearby edge devices, not just one

Filters & summarizes

Cuts noise and sends the cloud only what it truly needs

Adds local storage

Buffers data for minutes to hours if the link to the cloud drops

Runs shared services

A local gateway, security check or shared model for a whole site

Edge vs Fog — not the same layer

	EDGE	FOG
Scope	One device	Many nearby devices
Location	On or inside the device	A local server, gateway or router
Example	ESP32 reading its sensor	Gateway serving 50 machines
Job	Instant local decision	Aggregate, filter, buffer for the site

Fog computing scales edge ideas from one device up to an entire site, floor or campus.

How the pieces stack

A simple mental model for organizing Topic 4.

Device

Sense and act — the ESP32, Arduino or Pi from Topic 2

What is being measured or controlled?

Edge

Process locally — real-time control, Edge AI inference

What must be decided instantly, on the spot?

Fog

Aggregate nearby — a site-level gateway or local server

What can wait a few seconds, but not minutes?

Cloud

Store and learn — long-term analytics from Topic 3

What benefits from seeing the whole fleet?

A reaching model, not a strict standard — real systems blur these boundaries, but the split keeps them clear.

The Unit III roadmap

6 topics, 180 minutes.



1. **Introduction**
2. **Hardware Platforms**
3. **Cloud Computing**
4. **Edge Computing** — You are here
5. **IoT + AI**
6. **Recap & Assessment**

You are here: Topic 4. Real-time processing and Edge AI get the most depth.

By the end of this topic

You will be able to:



Explain

Why moving processing to the edge solves latency, bandwidth and reliability problems that the cloud alone cannot.



Compare

Weigh edge against cloud processing for a given application, and justify where each one fits.



Place

Position a fog node correctly between edge devices and the cloud, and describe its job.



Recognize

Identify Edge AI and TinyML in a system description, and see why the model has to be compressed first.

Look at any IoT system and ask: which decisions here truly cannot wait for the cloud?

The big picture

The need	Latency, bandwidth cost, unreliable links and privacy all push processing toward the device.
Edge vs Cloud	Edge is fast, offline-capable and private; cloud has the power, history and the global view.
Real-time	Hard vs soft deadlines — missing one is either unsafe or just lower quality.
Edge AI	Train big in the cloud, compress, deploy small, then infer instantly on the device (TinyML).
Fog computing	The middle tier that aggregates, filters and buffers for a whole site, not just one device.
Up next	Topic 5: IoT + AI — anomaly detection, predictive maintenance and computer vision.

One-line takeaway

> *“Edge computing does not replace the cloud — it answers the questions that cannot wait, right where the data is born.”*

Need for edge | Edge vs Cloud | Real-time | Edge AI | Fog computing

Up next: Topic 5 — IoT + AI: anomaly detection, predictive maintenance, computer vision and the full sensor-to-actuator pipeline.



AI Turns Sensor Data Into Decisions

~3 min

A raw stream of numbers becomes useful only when something learns to recognize what the numbers mean.

Patterns over raw numbers | From data to action

In Simple Words

A single sensor reading means little on its own. AI looks at many readings over time and learns what "normal" looks like.

Once it knows normal, it can spot the unusual, forecast what comes next, and even read an image the way a person would.



This same loop repeats across every section today — only the model and the data change.

45 Minutes, 5 Ideas

ML with sensor data — ~8 min

What makes sensor streams different, and the workflow used to model them.

Anomaly detection — ~10 min

How a model learns "normal" and flags what is not, with real examples.

Predictive maintenance — ~7 min

Turning an anomaly into an early warning, and the three maintenance styles.

Vision & intelligent systems — ~8 min

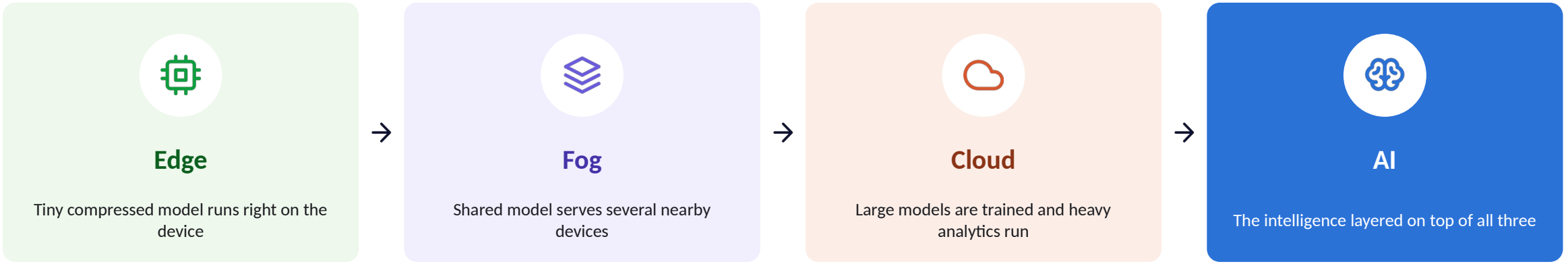
Cameras as sensors, and what makes a system genuinely intelligent.

The pacing below follows this same budget — deeper subtopics get more slides, not a fixed count.

1 Recap: where AI fits in the pipeline

~3 min

Topic 4 placed Edge, Fog and Cloud. AI can run at any of these layers.



Topic 5 crosses every layer

Topic 4 was about where computing physically happens. This topic is about what runs there: machine learning on sensor streams, anomaly detection, predictive maintenance, computer vision and the intelligent systems that tie sensing and action together.

The same model can live on a chip, on a gateway, or in the cloud — the question is only how big it needs to be.

2 Machine Learning with Sensor Data

~4 min

Sensor streams are time-series — sequences of readings, not single snapshots.

Time-series

Order matters — a reading only makes sense next to the ones that came before it. Shuffle the sequence and the pattern is lost.

Noisy

Real sensors drift with temperature, glitch occasionally, and pick up electrical interference. The model has to see past that.

Continuous

Data never stops arriving. A model deployed on a device has to keep up with a live stream, not a fixed dataset.

Multi-sensor

Temperature, vibration and current together often reveal a fault that any single sensor alone would miss.

None of these four traits are unique to IoT alone — together, they are why sensor ML is its own discipline.

3 The Machine Learning Workflow

~4 min

The same four-step process turns raw readings into a working model.

1 Collect & label

Gather historical readings, tagged normal or faulty where possible

→

2 Extract features

Turn raw readings into signals a model can learn from — averages, peaks, trends

→

3 Train & validate

Fit a model, then test it on data it has never seen before

→

4 Deploy & monitor

Run it on live data and keep checking that it still performs well

Why "validate" is its own step

A model can look accurate on the data it was trained on simply by memorizing it. Testing it on fresh, unseen readings is the only way to know it has actually learned the underlying pattern rather than the noise in one batch of data. This same collect-extract-train-deploy loop repeats for anomaly detection, predictive maintenance and computer vision — only the labels and features change.

Everything later in this topic is this same workflow, applied to a specific problem.

4 Anomaly Detection — How It Works

~4 min

Teach the model what normal looks like, then flag anything that does not fit.

Learn normal

The model studies weeks of healthy readings and builds a picture of the expected range for every sensor.

Watch live data

Every new reading is compared against that expected range continuously, in real time, as it arrives.

Flag the outlier

A reading far outside the normal pattern is marked as an anomaly and sent for review or an alert.

An anomaly is a signal, not a verdict

It means something changed — it still takes a person or a follow-up rule to decide whether that change is a fault, a new normal, or nothing at all.

Next: what this looks like in real IoT deployments.

5 Anomaly Detection — Where It Shows Up

~3 min

The same idea, applied across very different sensors.

Industrial motor

A pump vibrating slightly more than usual, hours before a bearing would visibly fail.

Cold-chain fridge

A commercial fridge that stops cycling on its normal schedule, risking spoiled stock.

Healthcare wearable

A heart-rate sensor reading an irregular rhythm outside the wearer's normal pattern.

Network traffic

Device traffic spiking at 3 a.m., a possible sign of a compromised IoT device.

Four different sensors, four different industries — the same "learn normal, flag the rest" idea underneath.

Why catching it early pays off

~3 min

Days–weeks

typical early-warning window

before a real failure occurs

~3–5x

costlier reactive repair

compared to a planned fix

Hours

unplanned downtime avoided

per prevented failure

Why this matters for maintenance

A machine that breaks down unannounced stops a whole line, not just itself — repairs are rushed, parts are ordered at short notice, and the schedule around it slips too. A model that spots the early drift in vibration, temperature or current gives the maintenance team days or weeks of lead time instead of zero. The fix becomes a scheduled task instead of an emergency, which is exactly the shift predictive maintenance is built to deliver.

This is the economic case for predictive maintenance, covered next.

6 Predictive Maintenance — The Loop

~4 min

Forecast a failure before it happens, instead of reacting after it does.

Monitor

Vibration, temperature and current are tracked continuously

→

Detect drift

Model notices the trend moving away from healthy behavior

→

Predict

Estimates a likely failure window, not just "something is wrong"

→

Schedule fix

Maintenance is planned before the machine actually stops

It builds directly on anomaly detection

Anomaly detection answers "is this different?" Predictive maintenance goes one step further and answers "different, heading which way, and by when?" That extra step — turning a flag into a forecast — is what lets a maintenance team schedule a fix around their own calendar instead of an emergency.

Not every deployment needs this extra step — next, when it is worth it.

7 Three Ways to Maintain a Machine

~3 min

Predictive is one option among three — each with its own cost and risk.

Reactive

Fix it only after it breaks.

Cheapest to plan for

Most disruptive when it fails

Preventive

Fix it on a fixed schedule, whether it needs it or not.

Predictable and safe

Wastes parts that were still good

Predictive

Fix it exactly when the data says it is starting to fail.

Minimal waste, minimal surprise

Needs data, sensors and a trained model

Most real fleets mix all three — predictive where sensors justify it, preventive elsewhere.

8 Computer Vision in IoT

~4 min

A camera turns "seeing" into a sensor reading a model can act on.

What the model looks for

Classification

Is this a ripe tomato or an unripe one?

Object detection

Where exactly are the people in this frame?

Defect spotting

Is there a crack, dent or stain on this part?

Counting & tracking

How many vehicles passed, and where did they go?

Why it usually runs at the edge

Video is heavy

Streaming raw video to the cloud for every camera does not scale — process it on-device instead

Privacy by design

A face can be counted or blurred locally, so the raw image never has to leave the site

Instant response

A safety camera on a production line must react in the same frame, not after a network round trip

Computer vision is Edge AI from Topic 4 applied to images instead of single numeric readings.

9 Intelligent IoT Systems

~4 min

Sensing, AI and action combine into a system that decides for itself.

What makes a system "intelligent"

Senses continuously

Never stops gathering the data it needs to decide

Interprets, not just reports

Turns raw readings into a judgment, not just a number

Acts without waiting

Closes the loop back to an actuator on its own

Improves over time

Retrained periodically as more data becomes available

From plain IoT to intelligent IoT

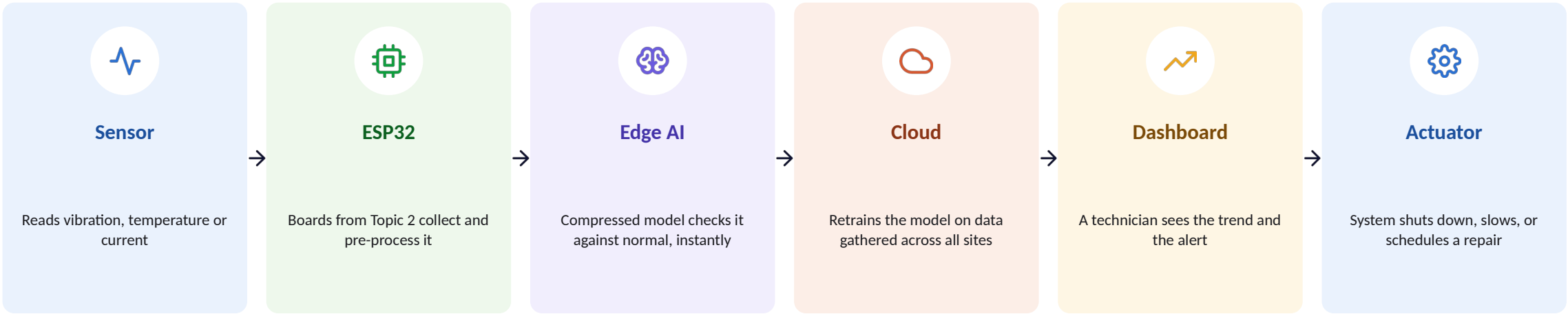
	PLAIN	INTELLIGENT
Reads	Shows the number	Explains what it means
Reacts	Waits for a person to decide	Decides and acts on its own
Learns	Same rules forever	Improves as new data arrives
Example	A thermostat with a fixed setting	A thermostat that learns your routine

Intelligent IoT is not a new layer — it is the payoff of combining everything from Units I to III.

The Full Pipeline, Walked Through

~4 min

Sensor → ESP32 → Edge AI → Cloud → Dashboard → Actuator, from start to finish.



Reading it end to end

A motor's vibration sensor feeds an ESP32, which runs a small Edge AI model checking for early signs of bearing wear, in real time, with no cloud round trip. Meanwhile the cloud aggregates this data with every other motor on the site to keep improving that model. A technician sees the trend on a dashboard and, if the pattern crosses a threshold, an actuator slows or stops the motor automatically — the same closed loop from Unit I, now powered by AI at every stage.

The Unit III roadmap

6 topics, 180 minutes — this topic's 45 minutes are the longest single block.



1. **Introduction**
2. **Hardware Platforms**
3. **Cloud Computing**
4. **Edge Computing**
5. **IoT + AI** — You are here
6. **Recap & Assessment**

You are here: Topic 5. Nine numbered sections across 13 content slides reflect that extra time.

By the end of this topic

You will be able to:



Describe

How machine learning turns a raw, noisy sensor stream into a usable signal.



Distinguish

Anomaly detection from predictive maintenance, and explain how one builds on the other.



Explain

Why computer vision in IoT usually runs at the edge, and what tasks it can perform.



Trace

The full pipeline from sensor to actuator, and where AI sits at each stage.

Look at any IoT dashboard and ask: is this just reporting a number, or actually understanding it?

The big picture

ML with sensor data	Time-series, noisy, continuous, multi-sensor — and the collect → extract → train → deploy workflow behind every model.
Anomaly detection	Learn what normal looks like, watch live data, flag what falls outside it — a signal, not a verdict.
Predictive maintenance	Anomaly detection plus a time estimate. One of three styles: reactive, preventive or predictive.
Computer vision	Cameras plus models classify, detect and count — usually processed at the edge for speed and privacy.
Intelligent systems	Sense, interpret, act and improve on their own, closing the loop from Unit I.
Up next	Topic 6: Recap & Self-Assessment — a platform comparison and Edge vs Cloud review for all of Unit III.

One-line takeaway

> *“AI is what lets an IoT system stop reporting numbers and start understanding them — and act before a problem becomes a failure.”*

ML on sensor data | Anomaly detection | Predictive maintenance | Computer vision | Intelligent systems

Up next: Topic 6 — Recap & Self-Assessment: platform comparison, Edge vs Cloud decisions, and a preview of Unit IV.



Closing the Loop on Unit III

~2 min

No new material — this topic checks what stuck across the last five topics before Unit IV begins.

1 Introduction

2 Hardware Platforms

3 Cloud Computing

4 Edge Computing

5 IoT + AI

Three things this recap does

Compares the platforms

Arduino, ESP32, Raspberry Pi and NodeMCU, side by side — from Topic 2.

Sharpens the Edge vs Cloud call

A short checklist for deciding where processing should actually happen.

Checks your understanding

Five quick questions covering Topics 1 through 5, then a look ahead to Unit IV.

Fifteen focused minutes — the shortest topic in the unit, and the fastest-moving.

1 Platform Comparison Recap

~5 min

Arduino, ESP32, Raspberry Pi and NodeMCU — from Topic 2, side by side.

Platform	Processing	Connectivity	Power use	Best for
Arduino	Basic microcontroller, no OS	None built-in — needs a shield	Very low	Simple sensing & control loops
ESP32	Dual-core, moderate power	Wi-Fi + Bluetooth built-in	Low	Connected sensors, Edge AI
Raspberry Pi	Full small computer, runs Linux	Wi-Fi, Bluetooth, Ethernet	Higher	Gateways, vision, heavier compute
NodeMCU	Microcontroller, Wi-Fi focused	Wi-Fi built-in	Low	Quick Wi-Fi prototypes

None of these four is "best" — the right pick depends on power, connectivity and compute needs together.

2 Edge vs Cloud: The Decision Checklist

~4 min

Four quick questions from Topic 4, revisited as a practical guide.

Does the decision have a hard deadline?

→ **Yes → Push it to the Edge**

→ No → Cloud has time to help

Is the raw data huge or continuous (e.g. video)?

→ **Yes → Process locally first**

→ No → Sending it is fine

Can the site lose connectivity?

→ **Yes → Edge keeps it running**

→ No → Cloud dependency is safe

Is the data sensitive or private?

→ **Yes → Filter before it leaves**

→ No → Cloud storage is fine

A "yes" to any one of these is often reason enough to push at least part of the job to the edge.

Self-assessment: five quick questions

~3 min

1. Which board would you pick for a battery-powered Wi-Fi sensor that also needs to run a small Edge AI model?
2. A dashboard update can wait a second, but a safety shut-off cannot. Where should each decision run?
3. What is the difference between anomaly detection and predictive maintenance, in one sentence?
4. Why does computer vision in IoT usually run at the edge rather than in the cloud?
5. What turns a plain IoT device into an "intelligent" one?

Answers are not printed here on purpose — talk them through with a partner before checking Topics 2–5.

Unit III, the big picture

~1 min

Introduction	Hardware, cloud, edge and AI matter together — and where this unit fits in the pipeline.
Hardware Platforms	Arduino, ESP32, Raspberry Pi, NodeMCU and how sensors connect to each.
Cloud Computing	Device management, storage and analytics — the deep-history, global view.
Edge Computing	Latency, bandwidth and privacy push processing next to the device; Fog scales it to a site.
IoT + AI	ML on sensor data, anomaly detection, predictive maintenance, vision and intelligent systems.
This topic	Platform recap, Edge vs Cloud checklist, self-assessment, and a look at Unit IV.

Up Next: Unit IV

~1 min

IoT Security and Data Analytics — 180 minutes, starting with why breaches happen.

1

Introduction

Real-world breach examples set the stage

2

Security Fundamentals

Authentication, encryption, secure comms

3

Threats & Attacks

Spoofing, DoS, malware, data leakage

4

Data Analytics

Time-series, visualization, AI/ML analytics

5

Blockchain + IoT

Decentralized IoT, smart contracts, identity

6

Recap & Assessment

Security checklist and analytics pipeline recap

Every layer secured in Unit IV is a layer this unit just taught you to build.

One-line takeaway

> *“The right platform, the right layer, the right model — Unit III was about making three good choices, not one big one.”*

Hardware platforms | Cloud computing | Edge computing | IoT + AI | Recap & assessment

Up next: Unit IV — IoT Security and Data Analytics, starting with why breaches happen in the first place.



1 Platforms turn data into action

Sense it, move it — now run it, store it, decide, and act on it

In Simple Words

Once Unit II gets a reading to a gateway or the cloud, something has to actually run the code, store the data, decide what matters, and turn that insight back into action in the physical world. That “something” is a platform — a board, a cloud service, an edge process, or a model — and Unit III is about choosing the right one.

Choosing where to compute, and what runs there, really matters for a system’s success.

What Makes It Hard?

Tiny compute budgets

Many boards have kilobytes of memory, not gigabytes.

Real-time limits

Some decisions can’t wait for a round trip to the cloud.

Cost and scale

A cloud bill multiplies fast across thousands of devices.

Trust in AI output

A model’s decision has to be reliable enough to act on.

2 Recap: where platforms live

Unit I’s five-layer architecture, with Processing, Application and Business in focus

5 Business

Judges whether the deployment is succeeding

4 Application

Delivers value to the end user — dashboards, alerts, actions

3 Processing

Stores, analyzes and pre-processes the data — edge or cloud

2 Network

Transmits data to wherever it is processed — Unit II’s job

1 Perception

Sensors and actuators perceive, and act on, the world



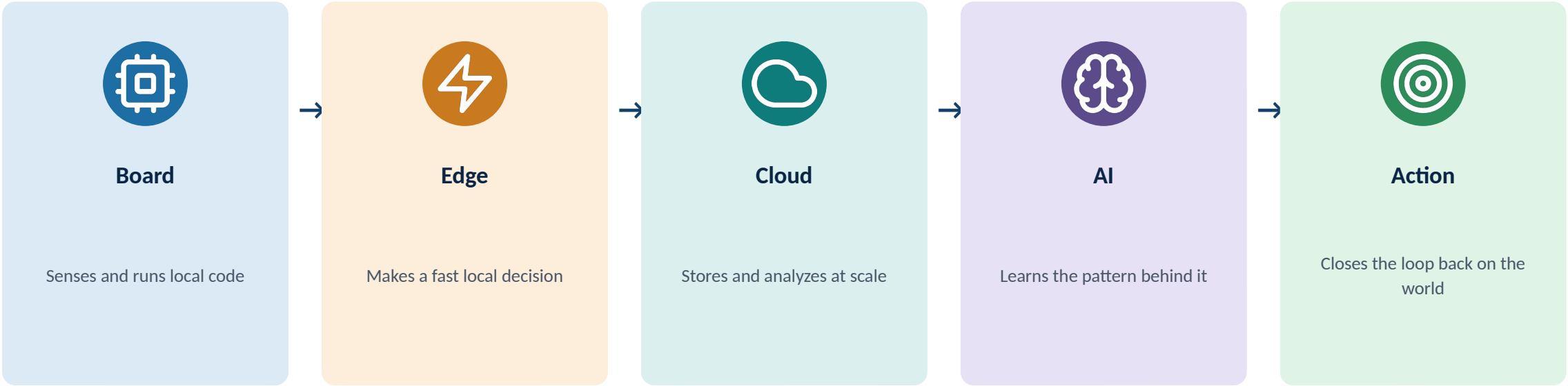
Unit III lives in Processing, Application and Business

Unit II’s whole job was carrying data to wherever it gets processed. Unit III is about that “wherever”: what hardware or cloud actually does the processing, where intelligence lives, and how a decision becomes an action again.

3

The pipeline runs both ways

Readings travel up to insight; decisions travel back down to actuators



The closed loop from Unit I: sense the world, decide what to do, then act on it — and every arrow above is a platform choice.

Board → Edge: what runs locally? Edge → Cloud: what's worth sending on? Cloud → AI: what pattern matters? AI → Action: who presses "go"?

4 Why choosing where to compute is hard

One of the highest-stakes engineering decisions in practical IoT



Tiny hardware budgets

A microcontroller may have kilobytes of RAM — not every algorithm fits.



Real-time limits

A safety stop can’t wait 200ms for a cloud round trip.



Cost at scale

Cloud compute and storage costs multiply across every device.



Trust and explainability

An AI decision has to be reliable enough to act on unsupervised.



Heterogeneous stacks

Different boards, clouds, and edge frameworks rarely match.

This is exactly why an entire unit is dedicated to hardware platforms, cloud, edge and AI.

5 Latency shapes every choice

A preview of the edge-versus-cloud trade-off we return to in Topic 4



Example: a conveyor-belt safety stop that must trigger inside 50 milliseconds simply cannot wait on a cloud round trip — it has to run on the board or at the edge. A weekly maintenance report, on the other hand, can happily wait on the cloud.

The lesson to carry forward: how fast a decision is needed shapes where it has to run. Expect this theme again and again.

6 One deployment, many platforms

Heterogeneity and scale, again — now one level up the stack

Heterogeneity: everything is different



Arduino / ESP32 / Pi

Bare-metal code to a full Linux OS



AWS / Azure / GCP

Different services, same job



TF Lite vs cloud ML

A model can run on-device or centrally

Unlike a corporate network of similar laptops, one IoT deployment can mix all of these.

Scale: two extremes



A single board

A hobby project prototyped on one ESP32 on a desk.



Thousands of edge nodes

An industrial site with thousands of sensors, a fleet of edge gateways, and one cloud dashboard.

Whatever we choose must work sensibly at both extremes.

7

Four ways to look at platforms

Unit III is organized around four questions

Hardware

What runs the code on the device?

- Arduino
- ESP32
- Raspberry Pi
- NodeMCU
- Sensors & modules

Cloud

Where does data live and get analyzed at scale?

- IoT cloud architecture
- Device management
- Data storage
- Cloud analytics
- IoT dashboards

Edge

What gets decided locally, right now?

- Need for edge
- Edge vs cloud
- Real-time processing
- Edge AI
- Fog computing

AI

How does the system get smarter over time?

- ML with sensor data
- Anomaly detection
- Predictive maintenance
- Computer vision
- Intelligent systems

8 Edge versus cloud

The same device data, processed in very different places

Edge options

Close to the device

- Runs on-device or on a nearby gateway
- Millisecond decisions, works offline
- Limited memory and compute power

Cloud options

Centralized and powerful

- Runs in a distant data center
- Massive compute, storage, and history
- Depends on a live connection, adds latency

Four dimensions to compare

Latency | Compute power | Cost | Connectivity dependence

Each has its place; there is rarely one best answer — Topic 4 goes much deeper.

9

How the pieces stack

A simple mental model for organizing Unit III



Hardware

Senses the world and runs the code, physically

What runs on the device?

Arduino, ESP32, Raspberry Pi, NodeMCU



Edge

Makes the fast decisions that can’t wait

What gets decided nearby?

Real-time processing, edge AI, fog computing



Cloud

Stores, aggregates and analyzes at scale

Where does the big picture live?

Device management, storage, dashboards



AI

Learns the pattern behind all of it

How does the system get smarter?

Anomaly detection, predictive maintenance

A teaching model, not a strict standard. Real systems blur these boundaries, but the split keeps Unit III clear.

10

The Unit III roadmap

Six topics, 180 minutes

10 min

1

Introduction

Why hardware + cloud + edge + AI matter together

You are here

40 min

2

Hardware

Arduino, ESP32, Raspberry Pi, NodeMCU, sensors

35 min

3

Cloud

IoT cloud architecture, device management, analytics

35 min

4

Edge

Edge vs cloud, real-time processing, edge AI, fog

45 min

5

IoT + AI

ML with sensor data, anomaly detection, computer vision

10 min

6

Recap

Platform comparison, self-assessment, Unit IV preview

You are here: topic 1, the introduction. IoT + AI gets the most time, hardware close behind.

11

By the end of Unit III, you will be able to…

Four verbs, four skills



Choose

Pick the right board — Arduino, ESP32, Pi, or NodeMCU — for a project's constraints.



Explain

How edge and cloud processing trade off against each other, and why.



Place

Where AI fits in a sensor-to-action pipeline, and what it needs to work.



Compare

Platforms and approaches side by side, using the same few dimensions.

The big picture

Unit III in five rows: what actually runs the code and closes the loop

The backbone	Unit II carries the data; Unit III is what does something with it.
Hardware	Boards and modules — what actually runs the code and senses the world.
Edge & Cloud	The recurring trade-off: fast and local, or powerful and central.
AI	Learns the pattern, so the system gets smarter over time.
Up next	Topic 2: IoT Hardware Platforms — Arduino, ESP32, Raspberry Pi, NodeMCU.



One-line takeaway

“Unit III turns data into action: the right board, the right cloud, the right edge, and a touch of AI — each in its place.”

Hardware | Cloud | Edge | AI

Up next: Topic 2 — IoT Hardware Platforms (Arduino, ESP32, Raspberry Pi, NodeMCU)